

2024 Information Retrieval and Extraction HW1

312553024 江尚軒

Questions

1. What kind of pre-processing did you apply to the document data or question text? Additionally, please discuss how different preprocessing methods affected the performance of the models?

In this project, I applied several preprocessing steps to both document and query text to enhance model performance:

1. **Remove HTML Tags:** Stripped HTML to keep only the core content.
2. **Lowercase Conversion:** Standardized all text to lowercase to reduce vocabulary size.
3. **Remove Punctuation/Special Characters:** Cleaned text for better tokenization.
4. **Stopword Removal:** Removed common words (e.g., "is," "the") to emphasize meaningful content.
5. **Stemming and Lemmatization:** Applied stemming (using Porter Stemmer) and lemmatization to reduce words to their root forms, e.g., "running" to "run."
6. **Remove Numbers:** Removed numeric data to focus on text features.
7. **Remove Extra Spaces:** Ensured compact and clean text.

Additionally, I found that Removing HTML Tags, Lowercase Conversion, Punctuation/Special Characters, Stopword Removal, Stemming, and Lemmatization could improve the performance. On the other hand, Removing Numbers and Extra Spaces would reduce the performance.

```
def preprocess_text(text):  
    # 1. Remove HTML tags  
    text = re.sub(r'<.*?>', '', text)  
  
    # 2. Convert to lowercase  
    text = text.lower()  
  
    # 3. Remove punctuation and special characters  
    text = re.sub(r'^\w\s', '', text)  
  
    # 4. Remove stopwords  
    stop_words = set(stopwords.words('english'))  
    text = ' '.join([word for word in text.split() if word not in stop_words])  
  
    # 5. Stemming  
    stemmer = PorterStemmer()  
    text = ' '.join([stemmer.stem(word) for word in text.split()])  
  
    # 6. Lemmatization  
    lemmatizer = WordNetLemmatizer()  
    text = ' '.join([lemmatizer.lemmatize(word) for word in text.split()])  
  
    # 7. Remove numbers  
    text = re.sub(r'\d+', '', text)  
  
    # 8. Remove extra spaces  
    text = ' '.join(text.split())  
  
    return text
```

2. Please provide details on how you implemented the vector model and BM25.

In this project, I implemented two models: **Vector Model** with **TF-IDF** and **BM25**.

TF-IDF:

1. Vocabulary and TF-IDF Calculation:

- Constructed a vocabulary from all unique words in the corpus.
- Computed **Term Frequency (TF)** for each document and **Inverse Document Frequency (IDF)** for each term to build weighted vectors.
- Constructed TF-IDF vectors for documents and queries.

2. Similarity Calculation:

- **Cosine similarity** was used to compare the query vector against document vectors.
- Retrieved top-3 documents by cosine similarity.

```
# Implement TFIDF
class TFIDF:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, corpus):
        self.corpus = [doc.split() for doc in corpus]
        self.vocab = list(set(word for doc in self.corpus for word in doc))
        self.vocab_index = {word: idx for idx, word in enumerate(self.vocab)}
        self.doc_vectors = self._calculate_tfidf_vectors()
```

BM25:

1. Parameters:

- **k1**: Controls term frequency scaling (I found that k1=1.75 is the best in this homework).
- **b**: Controls document length normalization (I found that b=1.0 is the best in this homework).

2. Scoring:

- Calculated **IDF** and document length adjusted **TF**.
- BM25 score was computed to rank the documents for each query.
- Retrieved top-3 documents by ranking scores.

```
# Implement BM25
class BM25:
    Tabnine | Edit | Test | Explain | Document | Ask
    def __init__(self, corpus, k1=1.5, b=0.75):
        self.corpus = [doc.split() for doc in corpus]
        self.k1 = k1
        self.b = b
        self.doc_lengths = np.array([len(doc) for doc in self.corpus])
        self.avg_doc_length = np.mean(self.doc_lengths)
        self.vocab = list(set(word for doc in self.corpus for word in doc))
        self.vocab_index = {word: idx for idx, word in enumerate(self.vocab)}
        self.doc_freqs = self._calculate_doc_freqs()
        self.idf_cache = self._calculate_idfs()
```

3. Compare the strengths and weaknesses of the vector model and BM25. What factors might account for the differences in their performance?

TF-IDF:

- **Strengths:** Simple to implement and easy to understand. Efficient for smaller datasets.
- **Weaknesses:** Lacks document length normalization, which can bias towards longer documents, and doesn't handle term frequency saturation well.

BM25:

- **Strengths:** Handles document length normalization effectively and controls term frequency saturation, making it adaptable and more reliable across varied datasets.
- **Weaknesses:** More complex with parameters (k_1 , b) that need tuning, which adds to computational overhead.

Factors Affecting Performance:

- **Document Length:** BM25's normalization gives it an advantage over TF-IDF, especially when documents vary significantly in length.
- **Term Frequency Saturation:** BM25 prevents overemphasis on repeated terms, whereas TF-IDF continues to increase term weights.
- **Adaptability:** BM25 is more flexible due to tunable parameters, whereas TF-IDF lacks this adaptability, making it less suited for diverse datasets.